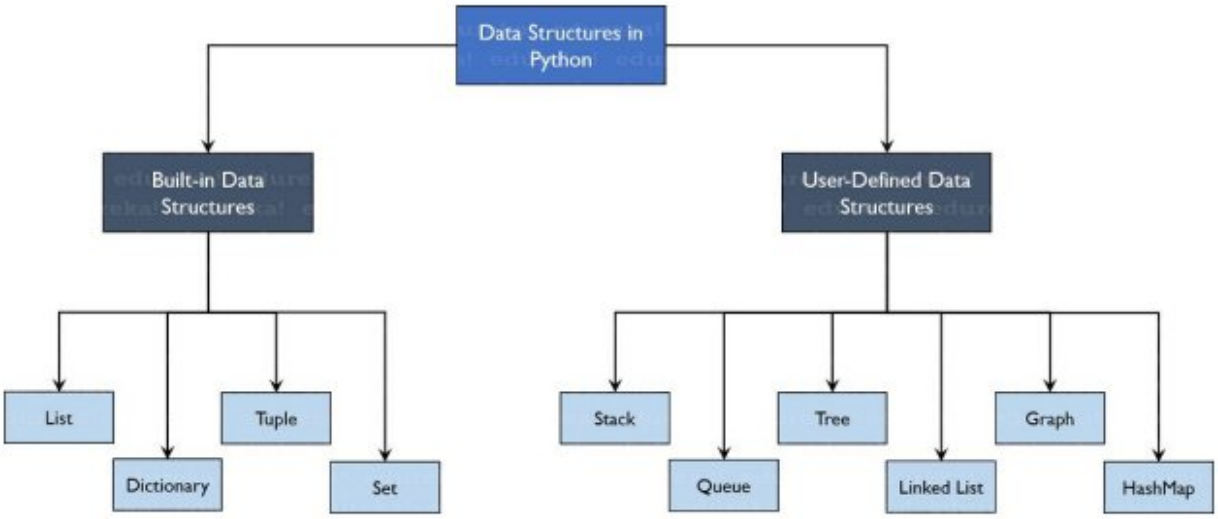
# PYTHON DATA STRUCTURE

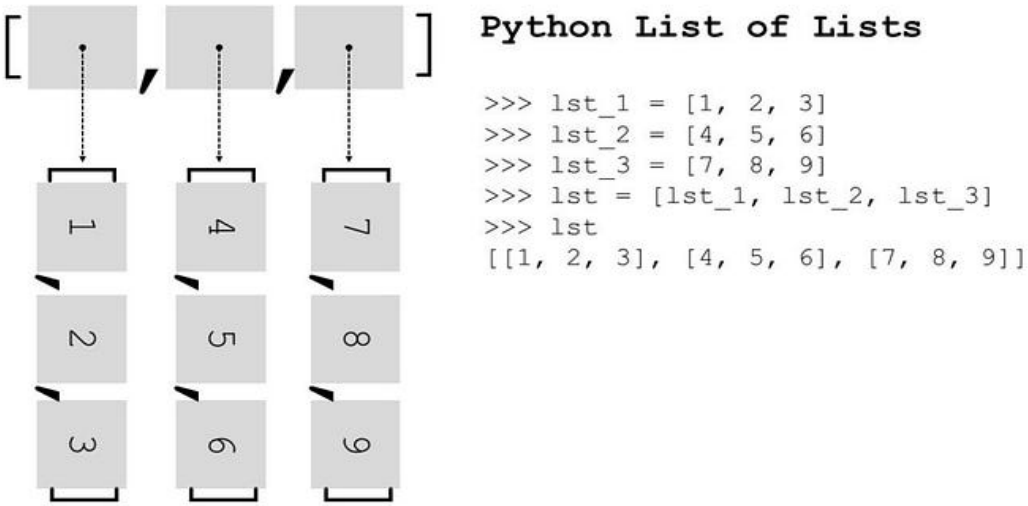
Pada pelajaran ini, kita akan memahami konsep struktur data yang digunakan saat satu variabel menyimpan banyak nilai. Sekarang, kita akan membahas tentang List, Tuple, Set, dan Dictionary.




Gambar 1. Data Structure in Python

## 1. LIST

List merupakan suatu jenis struktur data dalam bahasa pemrograman Python yang dapat menampung sejumlah data (objek/nilai), yang dikenal sebagai elemen list. Elemen-elemen tersebut disimpan dalam list berdasarkan urutan tertentu (Data Sequence).



## 1.1. Sifat List

### 1. Fleksibilitas Tipe Data Elemen List

Fleksibilitas python dimana isi sebuah list tidak harus memiliki tipe data yang sama. *List* dapat diisi dengan tipe data apa saja, string, integer, float, boolean, object, dan sebagainya.

```
listku = ["mahardika", 1, True, 4.6]
```

### 2. List Bersifat *Mutable* (Manipulasi)

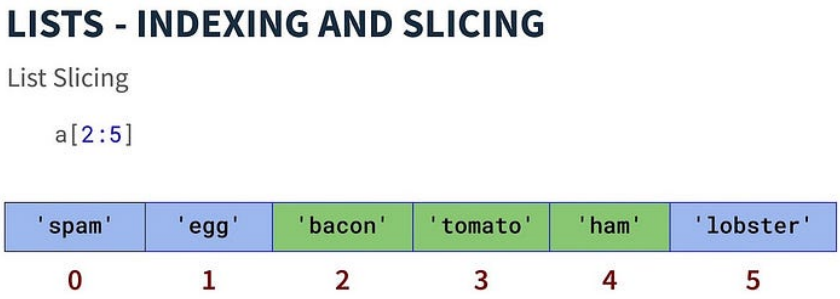
Isi sebuah list dapat dimanipulasi sehingga dapat berubah (*mutable*) yang artinya elemen list dapat diganti nilai elemen dengan cara ditambah maupun dikurangi.

| | "mahardika" | 1 | True | 4.6 |
|---|---|---|---|---|
| Index | 0 | 1 | 2 | 3 |
| Negative Index | -4 | -3 | -2 | -1 |

Untuk memanggil / menampilkan masing – masing nilai dalam list dengan cara sebagai berikut:

```
listku = ["mahardika", 1, True, 4.6]

listku[0]
'mahardika'

listku[1]
1

listku[2]
True

listku[3]
4.6
```

3. **Slicing (Memotong) Elemen dari List**




Contoh latihan :

# Kita punya list warna

warna = ["merah", "hijau", "kuning", "biru", "pink", "ungu"]

# Kita potong dari indeks ke-1 sampai ke-5

*print*(warna[1:5])

Maka hasil slicing index sebagai berikut:

```
warna = ["merah", "hijau", "kuning", "biru", "pink", "ungu"]
print(warna[1:5])
```

```
['hijau', 'kuning', 'biru', 'pink']
```

Perlu diperhatikan bahwa slicing dalam python, urutan terakhir tidak dibaca oleh bahasa python itu sendiri. Maka penulisan [1:5] memberikan perintah slicing mulai index 1 sampai index 5 sebagai batas akhir (**di mana index 5 ini tidak termasuk ke dalamnya**). **Sehingga yang dieksekusi adalah index ke 1 sampai 4.**

Jika ingin mengambil data dengan index terakhir tidak ikut dapat menggunakan perintah berikut:

List = [ 0, 1, 2, 3, 4, 5]

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|

List[0] = 0

List[1] = 1

List[2] = 2

List[3] = 3

List[4] = 4

List[5] = 5

List[0:] = [0,1,2,3,4,5]

List[:] = [0,1,2,3,4,5]

List[2:4] = [2, 3]

List[1:3]  = [1, 2]

List[:4] = [0, 1, 2, 3]

```
#data listtinggi_badan =
            [162, # index 0
             177, # index 1
             182, # index 2
             150, # index 3
             166, # index 4]print(tinggi_badan[:4])
```

Hasil output sebagai berikut:

```
tinggi_badan =[162, 177, 182, 150, 166]
print(tinggi_badan[:4])
```

```
[162, 177, 182, 150]
```

Jika ingin hanya mengambil data dengan index tengah-tengah (contoh index 2 sampai index 4) bisa menggunakan perintah berikut:

```
#data listtinggi_badan =
            [162, # index 0
```

```
            177, # index 1
            182, # index 2
            150, # index 3
            166, # index 4]# Mengambil data dengan index 2, 3, 4
print(tinggi_badan[2:5])
```

Hasil output sebagai berikut:

```
tinggi_badan =[162, 177, 182, 150, 166]
print(tinggi_badan[2:5])
```

```
[182, 150, 166]
```

Salah satu cara untuk mengubah daftar adalah dengan menetapkan indeks atau ekspresi irisan. Misalnya, jika kita ingin mengubah suhu ucup:

```
#data
suhu_keluarga_ucup = ['ayah ucup', 19, 'ucup', 19, 'ibu ucup', 20]

# mengubah suhu
suhu_keluarga_ucup[3] = [22]
print(suhu_keluarga_ucup)

#kita ingin mengganti nama 'ibu ucup' dengan 'mamah ucup', dan mengubah suhu ibu ucup menjadi 22

suhu_keluarga_ucup[-2:] = ['mamah ucup', 22]
print(suhu_keluarga_ucup)

#kita juga bisa menambahkan elemen baru menggunakan '+' operator. Jika kita ingin menambahkan adik ucup beserta suhunya sebagai berikut:

suhu_keluarga_ucup = suhu_keluarga_ucup + ['adik ucup', 20]
print(suhu_keluarga_ucup)
```

Maka hasil outputnya sebagai berikut:

```
suhu_keluarga_ucup = ['ayah ucup', 19, 'ucup', 19, 'ibu ucup', 20]
suhu_keluarga_ucup[3] = 22
print(suhu_keluarga_ucup)
```

```
['ayah ucup', 19, 'ucup', 22, 'ibu ucup', 20]
```

```
suhu_keluarga_ucup[-2:] = ['mamah ucup', 22]
print(suhu_keluarga_ucup)
```

```
['ayah ucup', 19, 'ucup', 22, 'mamah ucup', 22]
```

```
suhu_keluarga_ucup = suhu_keluarga_ucup + ['adik ucup', 20]
print(suhu_keluarga_ucup)
```

```
['ayah ucup', 19, 'ucup', 22, 'mamah ucup', 22, 'adik ucup', 20]
```

## 4. Jumlah Elemen pada List

Untuk mendapatkan jumlah elemen pada sebuah list dapat menggunakan fungsi *len()*.

```
tinggi_badan =
        [162, # index 0
         177, # index 1
         182, # index 2
         150, # index 3
         166, # index 4]
         len(tinggi_badan)
```

```
tinggi_badan =[162, 177, 182, 150, 166]
len(tinggi_badan)
```

```
5
```

## 5. Menambahkan Elemen List

Dalam menambahkan elemen dalam list, kita bisa menggunakan metode .append() ataupun .insert() seperti contoh berikut:

a. prepend(item) menambahkan item dari depan dan append(item) menambahkan item dari belakang

list.**append(x)** Tambahkan item ke akhir daftar *list*.

Setara dengan a[len(a):] = [x].

```
murid = ['Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani']



murid.append('Rizky')
murid.prepend('Rizky')



#result1:
['Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani', 'Rizky']



#result2:
['Rizky', 'Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani']
```

b. insert() untuk menambahkan item pada indeks tertentu.

list.**insert(i,x)**

Memasukkan item pada posisi tertentu. Argumen pertama adalah indeks elemen sebelum memasukkan, jadi a.insert(0, x) memasukkan di bagian depan daftar *list*, dan a.insert(len(a), x) sama dengan a.append(x).

```
murid = ['Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani']

murid.insert(2, 'Ahmad')

#result:
['Ayu', 'Bagus', 'Ahmad', 'Fajar', 'Cahyo', 'Ani', 'Rizky']
```

6. **Menghapus Elemen List**

Untuk mengurangi elemen sebuah list, kita dapat menggunakan *remove(), del, dan pop()*.

list.**remove(x)**

Hapus item pertama dari daftar *list* yang nilainya sama dengan *x*. Ini memunculkan ValueError jika tidak ada item seperti itu. *Method* remove() dengan paramter item yang akan dihapus.

list.**pop([i])**

Hapus item pada posisi yang diberikan dalam daftar, dan kembalikan. Jika tidak ada indeks yang ditentukan, a.pop() menghapus dan mengembalikan item terakhir dalam daftar.

```
# murid = ['Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani']
del murid[2] #artinya menghapus index ke-2 (Fajar)
#result:
['Ayu', 'Bagus', 'Cahyo', 'Ani']

# murid = ['Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani']
murid.pop(2)
#result:
['Ayu', 'Bagus', 'Cahyo', 'Ani']

# murid = ['Ayu', 'Bagus', 'Fajar', 'Cahyo', 'Ani']
murid.remove('Cahyo')
#result:
['Ayu', 'Bagus', 'Fajar', 'Ani']
```

## 7. Sort dan Reverse

‘sorted’ mengurutkan elemen list:

list.**sort**(*, *key=None*, *reverse=False*). Urutkan item daftar di tempat (argumen dapat digunakan untuk mengurutkan ubahsuaian *customization*, lihat sorted() untuk

penjelasannya). Elemen pada list dapat diurutkan dengan menggunakan fungsi *sort*(). Perintah sort akan mengurutkan dari nilai terkecil ke nilai terbesar.

list.**reverse**() Balikkan elemen daftar *list* di tempatnyafungsi *reverse()*, membalikkan urutan elemen pada list.

```
fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi']

fruits.reverse()#membalikan posisi dari belakang ke depan
['kiwi', 'banana', 'pear', 'apple', 'orange']

fruits.sort() #urutan alphabet
['apple', 'banana', 'kiwi', 'orange', 'pear']
```

‘sum’, tentu fungsi ini untuk menjumlahkan:

```
primes = [2, 3, 5, 7]
print(sum(primes))
>>>17
```

list.**copy**()

Kembalikan salinan daftar *list* yang dangkal. Setara dengan a[:]. ‘.copy()’, metode ini digunakan untuk men-copy suatu list ke dalam suatu list baru yang memiliki nilai yang sama. Dengan metode ini kita bisa melakukan manipulasi terhadap list baru tanpa mengubah nilai list aslinya.

```
a = [1,2,3,4,5]
b = a.copy()
print(a)
print(b)#lakukan manipulasi terhadap list b
```

```
b.append(6)
print(a)
print(b)

#result1:
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]

#result2:
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 6]
```

## 8. List Comprehension

List comperehension merupakan cara elegan untuk membuat list berdasarkan list yang ada. List comperehension membantu men-*generate* list menggunakan ekspresi tertentu. Fasilitas ini sangat bermanfaat saat kita ingin membuat list dengan n elemen yang langsung terisi berdasarkan pola tertentu.

```
number = [1,5,10,6,7,8,12]
result = [i for i in number if i>7]
result:
[10, 8, 12]

m = [x for x in range(10,101,10) ] # membuat list m berisi bilangan
kelipatan 10 < 100
print(m)
output:
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

n = [j*j for j in range(10)] # membuat list n berisi bilangan kuadrat
0-9
print(n)
output:
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

## 2. Tuple

Tuple juga merupakan struktur data yang digunakan untuk menyimpan satu atau lebih data di dalamnya. Perbedaanya terletak pada kemampuan untuk melakukan perubahan anggota. Anggota pada tuple tidak dapat diubah sebagian, akan tetapi dapat diubah keseluruhan dengan memanfaatkan melakukan inisiasi variabel dengan nilai yang baru. Inisiasi variabel tuple dilakukan dengan menggunakan tanda kurung ‘()’.

```
tuple_01 = (1,2,3,4)
```

- **Mengakses Elemen Tuple**

Untuk mengakses elemen tuple dapat menggunakan index positif dan negatif, sebagai contoh:

```
tuple_01 = (1,2,3,4)#mengakses index positif (dari depan)
print(tuple_01[1])
```

```
output :
2#mengakses index negatif (dari belakang)
print(tuple_01[-1])
output:
4
```

- **Tidak Dapat Mengubah dan Menambah Elemen Tuple**

Tidak seperti list dimana tidak dapat mengubah atau menambah tuple. Pembuktian sebagai berikut:

```
tuple_01[1] = 2TypeError: 'tuple' object does not support item assignment
```

Hal yang sama terjadi jika kita menggunakan metode append() untuk menambahkan anggota.

```
tuple_01.append(2)AttributeError: 'tuple' object has no attribute 'append'
```

- **Memotong dan Menduplikasi Tuple**

```
tuple_01 = (1,2,3,4)

tuple_03 = tuple_01[0:2] #memotong
tuple_04 = tuple_01[:]   #menduplikat

print(tuple_03)
(1, 2)

print(tuple_04)
(1, 2, 3, 4)
```

Kita dapat slice tuple seperti list dan juga melakukan duplikasi

- **Menginisiasi Multi-Variabel Menggunakan Tuple**

Walau kita tidak dapat melakukan perubahan pada tuple, tapi kita bisa menginisiasikan anggota tuple ke dalam variabel-variabel. Hanya saja variabel yang diinisiasikan harus berjumlah sama dengan anggota tuple tersebut.

```
tuple_01 = (1,2,3,4)

#sekarang akan kita masukkan kedalam 4 variabel dikarenakan tuple
tersebut memiliki 4 anggota
a, b, c, d = tuple_01

print(a, b, c, d)
print(type(a), type(b), type(c), type(d))

output:
1 2 3 4
<class 'int'> <class 'int'> <class 'int'> <class 'int'>
```

## Kelebihan Menggunakan Tuple?

Apa yang membedakan Tuple dengan List? Ada tiga poin utama antara lain:

1. Karena strukturnya lebih sederhana, struktur data tuple memakan memori lebih sedikit dibandingkan dengan list. Untuk data yang berukuran besar ini akan sangat menguntungkan.
2. Memori yang lebih sedikit juga mempengaruhi waktu proses dari tuple. Tuple memiliki waktu proses yang relatif lebih singkat daripada list.

3. Anggota yang tidak dapat diubah membuat tuple menjadi pilihan saat kita ingin menggunakannya untuk menyimpan suatu parameter yang tidak boleh berubah nilainya.

## 3. Set

Set dalam bahasa Indonesia adalah “himpunan”, merupakan struktur data yang dapat memiliki satu atau lebih anggota didalamnya.

- **Perbedan Set dengan List/Tuple**

Terdapat 2 perbedaan utama antara set dengan list atau tuple yaitu:

1. Set tidak memiliki koordinat/index tiap anggotanya, jadi untuk set posisi anggota dianggap tidak penting.
2. Set tidak dapat memiliki lebih dari satu anggota dengan nilai yang sama

- **Membuat/Inisiasi Set**

Untuk menginisiasi suatu variabel set ada dua cara yang dapat digunakan. Pertama dengan menggunakan tanda kurung kurawal “{}” atau dengan menggunakan metode set(). Metode set() biasanya digunakan untuk inisiasi set kosong atau untuk mengubah list/tuple menjadi set.

```
#Inisiasi Set
set_01 = {4,5,6,2}

print(set_01)
{2, 4, 5, 6} #output default telah berurutan

#inisiasi mengubah list menjadi set
set_02 = set()
set_03 = set([2,1,4,3])

print(type(set_02))
<class 'set'>

print(set_03)
{1, 2, 3, 4}
```

- **Mengakses Anggota Set**

Set merupakan struktur data yang tidak memiliki urutan atau index. Sehingga tidak dapat mengakses anggota dengan menggunakan index seperti pada list atau tuple.

```
set_01 = {4,5,6,2}
print(set_03[1])TypeError: 'set' object does not support indexing
```

- **Menghapus Anggota Set**

Karena tidak dapat memiliki index, maka untuk menghapus anggota set menggunakan dua metode yaitu discard() dan remove()

```
set_01 = {4,5,6,2}

#menggunakan discard()
set_01.discard(4) #Menghapus nilai 4 pada set
print(set_01)
{2, 5, 6}

#menggunakan remove()
set_01.remove(2)#Menghapus nilai 2 pada set dari hasil discard
print(set_01)
{5, 6}
```

### Menambah Anggota Set

Menambah anggota set dapat dilakukan dengan menggunakan metode add() yang digunakan untuk menambahkan hanya satu anggota pada set.

```
set_04 = {2,3,4,5,6}
set_04.add(1)

print(set_04)
{1, 2, 3, 4, 5, 6}
```

Untuk menambahkan lebih dari satu anggota pada set dapat menggunakan metode update(). Dengan metode update ini juga bisa memasukan list atau tuple sebagai bahan kita.

```
#Menambahkan/memasukan list
set_04 = {1, 2, 3, 4, 5, 6}
set_04.update([4,7,8])

print(set_04)
{1, 2, 3, 4, 5, 6, 7, 8}#nilai 4 tidak bisa double maka hanya sekali
```

- **Mengubah Anggota Set**

Dikarenakan set adalah struktur data yang tidak memiliki index sehingga tidak dapat mengubah secara langsung, namun dengan cara menghapus lalu menambahkan anggota yang diinginkan. Contoh mengganti nilai 4 menjadi 10 sebagai berikut:

```
set_04= {1, 2, 3, 4, 5, 6, 7, 8}
set_04.discard(4)#hapus nilai 4

print(set_01)
{1, 2, 3, 5, 6, 7, 8}
```

Lalu menambahkan 10 sebagai anggota set_04

```
set_04.add(10) #menambahkan nilai 10

print(set_04)
{1, 2, 3, 5, 6, 7, 8, 10}
```

**Operasi pada Set**

Sset adalah himpunan, artinya semua operasi yang bisa kita lakukan pada sebuah himpunan dapat kita lakukan pada set python.

Ada empat operasi yang dapat dilakukan pada set:

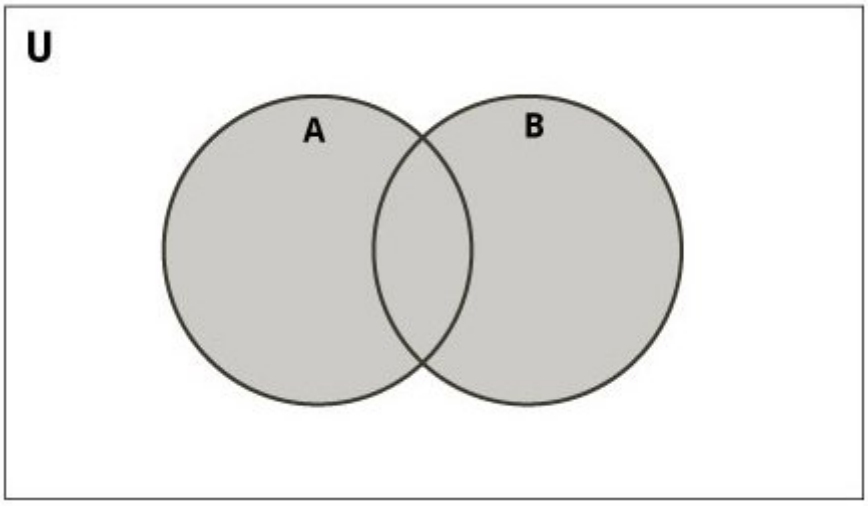
1. Union
2. Intersection
3. Difference
4. Symmetric Difference

Pembahasan satu persatu operasi sebagai berikut:

**1. Union**

Union adalah saat kita menggabungkan semua anggota set. Mempersiapkan dua set yaitu Set A dan Set B sebagai berikut:

```
Set_A = {1,2,3,4}
set_B = {3,4,5,6}
```

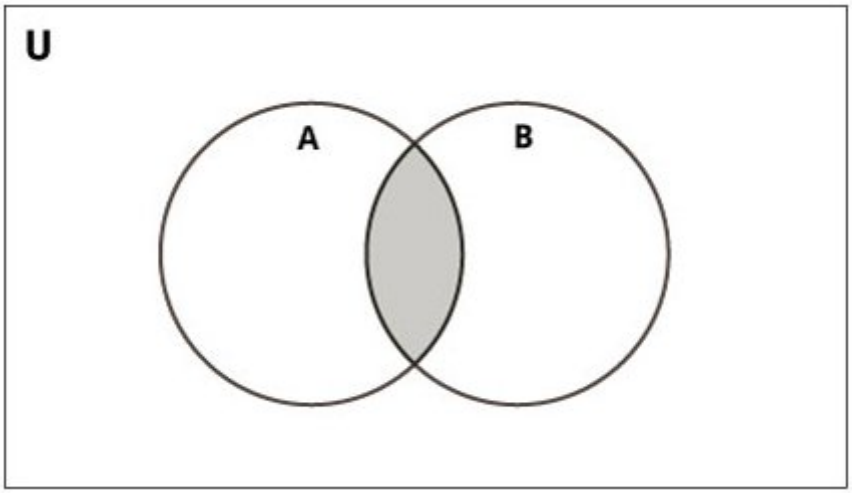



Union dapat dilakukan dengan menggunakan tanda “|” atau metode union()

```
print(set_A|set_B)
print(set_A.union(set_B))output:
{1, 2, 3, 4, 5, 6}
```

## 2. Intersection

Intersection adalah irisan, dimana kita mencari nilai irisan dari dua buah set.

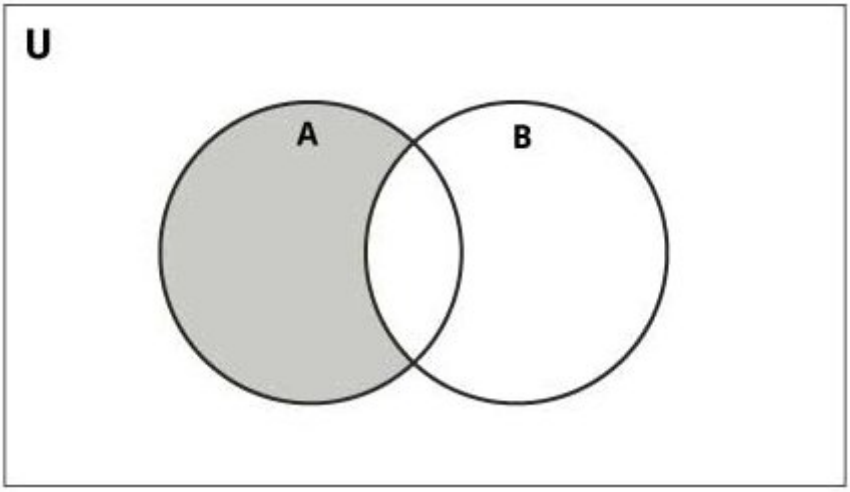



Intersection dapat dilakukan dengan menggunakan tanda “&” atau menggunakan metode intersection()

```
print(set_A & set_B)
print(set_A.intersection(set_B))output:
{3, 4}
```

## 3. Difference

Difference adalah operasi saat kita mencari perbedaan anggota dari dua buah set atau lebih.




Difference dapat dilakukan dengan menggunakan tanda “-” atau menggunakan metode difference().

- A — B

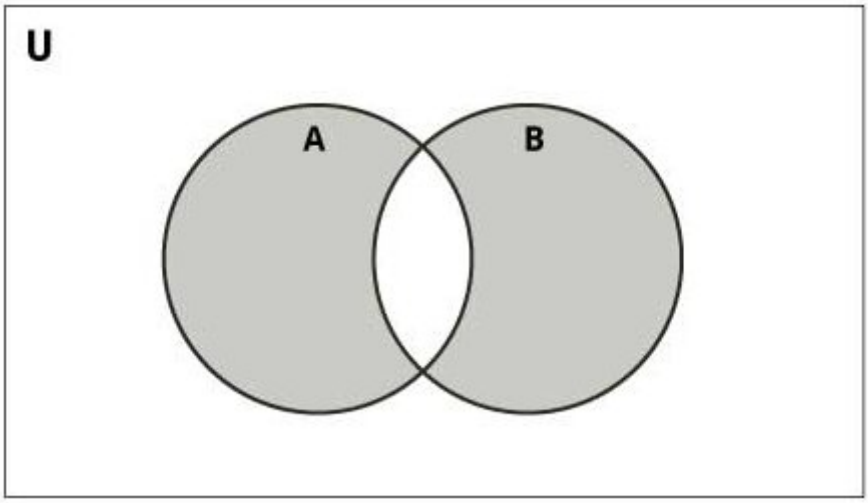
```
print(set_A - set_B)
print(set_A.difference(set_B))output:
{1, 2}
```

- B — A

```
print(set_B - set_A)
print(set_B.difference(set_A))output:
{5, 6}
```

### 4. Symmetric Difference

Symmetric Difference adalah semua anggota pada set A dan B dikurangi dengan anggota yang berada pada dua set tersebut.




symmetric difference dapat dilakukan dengan menggunakan tanda “^” atau menggunakan metode symmetric_difference().

```
print(set_A ^ set_B)
print(set_A.symmetric_difference(set_B))output:
{1, 2, 5, 6}
```